UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants. Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2014-06, page [2]

Unofficial model answer:

The Von Neumann architecture, as used in the IAS machine, consists of five main components: (1) Memory (Main Store): stores both program instructions and data in a single address space; it is organised as an array of words, each with a unique address, and holds 2^12 = 4096 words of 40 bits in the IAS machine. (2) Arithmetic Logic Unit (ALU): performs arithmetic operations (addition, subtraction, multiplication, division) and logical operations (AND, OR, NOT) on data fetched from memory; it contains an accumulator (AC) and a multiplier-quotient (MQ) register. (3) Control Unit (CU): fetches instructions from memory, decodes them, and generates control signals to coordinate the other components; it contains the instruction register (IR), program counter (PC), and memory address register (MAR). (4) Input/Output (I/O) Equipment: provides the means for the operator to enter programs and data and to receive results; in the IAS machine this was via paper tape, teletype, or magnetic tape. (5) Registers (as part of the CPU): temporary storage for operands, addresses, and instructions; key registers include the accumulator (AC), multiplier-quotient (MQ), instruction register (IR), program counter (PC), memory address register (MAR), and memory buffer register (MBR). A suitable diagram would show these components interconnected by buses: the CPU (containing CU, ALU, and registers) connected to memory and I/O via address, data, and control buses. The diagram should illustrate the stored-program concept where instructions and data reside together in memory.

Marking points:

• *1 mark: Memory – stores both instructions and data in a single address space; briefly describe its role.*

• *1 mark: ALU – performs arithmetic and logical operations; mention its role in data processing.*

• *1 mark: Control Unit – fetches, decodes, and executes instructions; coordinates other components.*

• *1 mark: Input/Output – provides interface for entering programs/data and receiving results.*

• *1 mark: Diagram – an appropriate block diagram showing the five components and their interconnections (buses) is provided.*

Model confidence: high.

Model note: The question refers to an 'appropriate diagram' but no figure is supplied in the extracted text; the answer describes the expected diagram. The IAS machine specifics (e.g., 4096 words of 40 bits) are included as illustrative detail but are not required for full marks. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 2 (5 marks)

Source: CSC429-2018-06, page [2]

Unofficial model answer:

a) Machine X is the **Difference Engine** (Charles Babbage's mechanical machine, later followed by his Analytical Engine). The clue 'solve four basic arithmetic functions' and 'improved from the Stepped Reckoner' points to Babbage's Difference Engine, the machine that is regarded as the first machine deserving to be called a computer.

b) The Difference Engine belongs to the **first generation** of computers (mechanical/electro-mechanical era, roughly 1642–1945), which covers early mechanical calculating devices such as the Pascaline, Stepped Reckoner and Babbage's engines.

c) The function of machine X is to **perform the four basic arithmetic operations (addition, subtraction, multiplication and division)** automatically by mechanical means, i.e. it was designed as an automatic mechanical calculator to compute and print mathematical/arithmetic tables.

© Hak Cipta Universiti Teknologi MARA

d) The machine invented prior to machine X is the **Stepped Reckoner** (Leibniz's stepped reckoner), an early mechanical calculator that influenced Babbage's design; the Pascaline (Pascal's calculator) is also an acceptable earlier mechanical calculator.

e) Because of his success in conceiving machine X, Charles Babbage is given the title **'Father of the Computer'**.

Marking points:

• 1 mark (a): Correctly names the Difference Engine (accept Analytical Engine only if clearly justified; Difference Engine is the expected answer).

• 1 mark (b): States first generation (mechanical era).

• 1 mark (c): States it performs the four basic arithmetic operations / automatic mechanical calculation.

• 1 mark (d): Names the Stepped Reckoner (accept Pascaline as an earlier mechanical calculator).

• 1 mark (e): States 'Father of the Computer'.

Model confidence: medium.

Model note: Figure 1 is supplied only as a caption/extracted text; the answer relies on the statements in the question text. The question wording ('solve four basic arithmetic functions', 'improved from Stepped Reckoner') is slightly loose historically, since the Difference Engine was designed for tabulating polynomials; the expected exam answer is nonetheless the Difference Engine. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 3 (5 marks)

Source: CSC429-2024-01, page [2]

Unofficial model answer:

a) Binary is used in computer systems because digital electronic circuits naturally have two stable states (e.g., high/low voltage, on/off), which map directly to bits 0 and 1. This makes binary representation reliable, simple to implement with transistors, and resistant to noise; decimal would require ten distinct voltage levels, increasing complexity, cost, and error rate.

b) In decimal representation, each digit (0–9) requires a separate storage element (vacuum tube). The value 139 has three decimal digits: '1', '3', and '9'. Therefore, the number of vacuum tubes needed is 3. Workflow: write 139 in decimal → count digits = 3 → each digit needs one tube → total = 3 tubes.

c) In binary representation, convert 139 to binary: 139 ÷ 2 = 69 remainder 1; 69 ÷ 2 = 34 remainder 1; 34 ÷ 2 = 17 remainder 0; 17 ÷ 2 = 8 remainder 1; 8 ÷ 2 = 4 remainder 0; 4 ÷ 2 = 2 remainder 0; 2 ÷ 2 = 1 remainder 0; 1 ÷ 2 = 0 remainder 1. Reading remainders bottom-up gives 10001011_2. This binary number has 8 bits, so 8 vacuum tubes are needed (one per bit). Workflow: repeated division by 2 → binary 10001011 → count bits = 8 → total = 8 tubes.

Marking points:

• 1 mark: (a) correct reason – binary matches two-state electronic devices / simpler and more reliable than decimal.

• 1 mark: (b) identifies that decimal 139 has 3 digits.

• 1 mark: (b) concludes 3 vacuum tubes needed with clear workflow.

• 1 mark: (c) correctly converts 139 to binary 10001011 (or equivalent) with shown division workflow.

• 1 mark: (c) counts 8 bits and concludes 8 vacuum tubes needed.

Model confidence: high.

Model note: Assumes one vacuum tube stores one decimal digit or one binary bit, as implied by the question context. No figure or table was provided; answer is based solely on the question text.

QUESTION 4 (5 marks)

Source: CSC429-2023-07, page [2]

Unofficial model answer:

a) A touch screen is a combined input/output device because it integrates a transparent sensing layer (e.g., capacitive or resistive) over a display panel. As an INPUT device, when a finger or stylus touches the screen, the sensing layer detects the position of contact and converts it into digital coordinate data that is sent to the processor, allowing the user to select icons, type on an on-screen keyboard, or draw. As an OUTPUT device, the underlying display (LCD/OLED) presents visual information such as text, images, and the on-screen keyboard back to the user. Thus the same physical unit both accepts data/commands from the user (input) and presents processed results to the user (output), making it a bidirectional I/O device.

b) One other device that can act as both input and output is a modem (or a network interface card/wireless network adapter): it receives data from the computer (output) and receives incoming data from the network to pass to the computer (input). Other acceptable examples: a headset with microphone, a printer/scanner multifunction device, or a USB flash drive (reads and writes data).

Marking points:

• 1 mark: states touch screen has a touch-sensing layer/panel that detects contact position

• 1 mark: explains input role - converts touch position into coordinate/signal data sent to processor for user commands

• 1 mark: explains output role - display panel shows visual information/results to the user

• 1 mark: concludes it is bidirectional because one unit both accepts user data and presents output

• 1 mark: names ONE valid bidirectional device (e.g., modem, NIC, headset with mic, multifunction printer, USB drive)

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. Part (b) accepts any device that genuinely performs both input and output; a plain keyboard or plain monitor alone would not qualify.

QUESTION 5 (10 marks)

Source: CSC429-2025-07, page [4]

Unofficial model answer:

a) SRAM stores each bit in a bistable latch (typically a 6-transistor cell: two cross-coupled inverters plus two access transistors). The cross-coupled inverters hold the bit as a stable state (Q and Q-bar); the access transistors connect the cell to the bitlines when the wordline is asserted, allowing read or write. DRAM stores each bit as charge on a capacitor (1T1C cell: one access transistor and one storage capacitor). A logic 1 is represented by charge stored on the capacitor and a logic 0 by little or no charge. Because the capacitor leaks, DRAM cells must be periodically refreshed (read and rewritten) to retain data. (Note: the question text says 'DFiAM'; this is taken to mean DRAM.)

b) SRAM is faster than DRAM because SRAM uses a bistable latch that drives the bitline actively during a read, so the bitline voltage swing develops quickly and no refresh or precharge-restore cycle is needed for retention. DRAM reads are destructive: the small charge on the capacitor must be sensed by a sensitive sense amplifier, then the cell must be rewritten (restored), and the bitline must be precharged before the next access. DRAM also has higher latency due to row activation, column access, and refresh overhead. Thus SRAM has lower access latency and higher speed.

c) Cost: SRAM is much more expensive per bit than DRAM because an SRAM cell uses about 6 transistors versus 1 transistor plus 1 capacitor for DRAM, so SRAM has lower density and larger area per bit. DRAM is therefore cheaper per bit and is used for main memory, while SRAM is used for caches. Power consumption: SRAM consumes relatively low static power (only leakage) when idle, but can consume dynamic power during active reads/writes. DRAM consumes static power due to refresh even when idle, and has higher active power from row activation and precharge; however, DRAM's per-bit power can be lower in some standby contexts. Overall, SRAM has lower idle/static power per cell but higher cost per bit; DRAM has refresh power overhead but lower cost per bit.

© Hak Cipta Universiti Teknologi MARA

Marking points:

• 1 mark: SRAM cell description (bistable latch / cross-coupled inverters / 6T).

• 1 mark: DRAM cell description (capacitor storage / 1T1C).

• 1 mark: DRAM refresh requirement mentioned.

• 1 mark: SRAM read is non-destructive and actively driven.

• 1 mark: DRAM read is destructive and requires sense amplifier and restore.

• 1 mark: DRAM requires precharge/row activation overhead.

• 1 mark: SRAM cell size (≈6 transistors) vs DRAM (1T1C) leading to density difference.

• 1 mark: SRAM higher cost per bit, DRAM lower cost per bit.

• 1 mark: SRAM low static power (leakage only) when idle.

• 1 mark: DRAM refresh power overhead and/or active power comparison.

Model confidence: high.

Model note: The question text contains 'DFiAM', which is assumed to be a typo for DRAM. No figure or table was supplied. The answer is an unofficial model answer, not an official UTM marking scheme.

QUESTION 6 (10 marks)

Source: CSC429-2026-02, page [5]

Unofficial model answer:

Part (a) — Completing the taxonomy (Figure 4). The taxonomy follows Flynn's classification at the top and then subdivides the MIMD branch by memory organization. The missing labels are: **(a) Single Instruction, Single Data (SISD)** — the left branch under 'Single instruction' (a single processor executing one instruction stream on one data stream). **(b) Uniprocessor** — the SISD leaf, i.e. a conventional single-processor machine. **(c) Shared Memory** — the MIMD branch that is subdivided into tightly coupled multiprocessors sharing memory. **(d) Multiple Instruction, Multiple Data (MIMD)** — the right branch under 'Multiple instruction'. **(e) Distributed Memory** — the loosely coupled MIMD branch (each processor has its own private memory, e.g. multicomputers/clusters). **(f) Symmetric Multiprocessor (SMP)** — the shared-memory leaf in which all processors have equal access to a single shared main memory. **(g) NUMA (Non-Uniform Memory Access)** — the other shared-memory leaf, where memory access time depends on which processor/memory module is accessed. So the completed tree reads: Processor Organization → SISD (Uniprocessor) and MIMD; MIMD → Shared Memory (SMP, NUMA) and Distributed Memory.

Part (b) — Two disadvantages of SMP. First, **memory contention / limited scalability**: because all processors share one main memory and a common bus or interconnect, simultaneous accesses contend for the same bandwidth, so adding processors yields diminishing returns and the system saturates (the bus/memory becomes the bottleneck). Second, **cache coherence overhead and synchronization cost**: with private caches per processor, hardware must keep cached copies consistent (snooping or directory protocols), which adds traffic and complexity, and shared-data programs require locking/synchronization that serializes execution and limits speed-up. (Acceptable alternatives: single point of failure in shared memory/bus, higher cost/complexity of coherent hardware, and difficulty of writing correct parallel software.)

Marking points:

• 1 mark: (a) SISD correctly placed under Single instruction

• 1 mark: (b) Uniprocessor correctly placed as the SISD leaf

• 1 mark: (d) MIMD correctly placed under Multiple instruction

• 1 mark: (c) Shared Memory correctly placed as the MIMD subdivision

• 1 mark: (e) Distributed Memory correctly placed as the loosely coupled MIMD branch

• 1 mark: (f) Symmetric (SMP) correctly placed under Shared Memory

• 1 mark: (g) NUMA correctly placed under Shared Memory

• 1 mark: first valid SMP disadvantage explained (e.g. memory/bus contention limiting scalability)

© Hak Cipta Universiti Teknologi MARA

• 1 mark: second valid SMP disadvantage explained (e.g. cache-coherence overhead / synchronization cost)

• 1 mark: clarity/accuracy of terminology across the answer (or split the two disadvantages as 1.5 marks each if the examiner prefers)

Model confidence: high.

Model note: Figure 4 was supplied only as a caption/crop description; the label letters (a)-(g) and their positions are inferred from the standard Stallings taxonomy of parallel processor architecture. If the actual figure numbers the boxes differently, map the same content labels to the corresponding letters. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 7 (10 marks)

Source: CSC429-2023-07, page [3]

Unofficial model answer:

a) Two bus types available in a computer system are the address bus and the data bus. The address bus carries memory or I/O addresses from the CPU to other components, while the data bus carries the actual data between the CPU, memory, and I/O devices. (Other acceptable answers include the control bus and the system bus.)

b) System interconnection (system bus) and CPU interconnection differ in two key characteristics. First, scope and connectivity: the system bus connects the CPU, main memory, and I/O devices and is shared by all system components, whereas CPU interconnection is internal to the processor and links the CPU components such as registers, ALU, and control unit. Second, speed and width: the system bus is typically slower and may be narrower because it must drive external devices over longer distances, while CPU interconnection is much faster and wider to support high internal bandwidth and low latency.

c) Data transfer timing refers to how the timing of data transfers between the CPU and memory or I/O devices is coordinated. There are two main types: synchronous timing and asynchronous timing. In synchronous timing, all bus operations are synchronized to a common clock signal; data transfers occur at fixed clock intervals, which is simple and fast for devices that can operate at the bus clock rate, but it forces all devices to run at the speed of the slowest device. In asynchronous timing, there is no common clock; instead, a handshake protocol (e.g., request/acknowledge signals) is used, allowing devices of different speeds to communicate efficiently, at the cost of more complex control logic and potentially lower performance. A third type, semi-synchronous timing, is a hybrid that uses a clock but allows some handshaking for slower devices.

Marking points:

• 1 mark: Correctly lists one bus type (e.g., address bus, data bus, control bus, system bus).

• 1 mark: Correctly lists a second bus type.

• 1 mark: Identifies one valid difference between system interconnection and CPU interconnection (e.g., scope/connectivity).

• 1 mark: Provides a clear explanation of that difference.

• 1 mark: Identifies a second valid difference (e.g., speed/width).

• 1 mark: Provides a clear explanation of that second difference.

• 1 mark: Names synchronous timing and explains it (common clock, fixed intervals).

• 1 mark: Names asynchronous timing and explains it (handshake, no common clock).

• 1 mark: Contrasts the two types (e.g., simplicity vs. flexibility, slowest-device limitation vs. mixed speeds).

• 1 mark: Mentions semi-synchronous or gives a relevant example/application of each timing type.

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify a figure or table, so no limitations apply. Part (a) accepts any two valid bus types; part (c) expects synchronous and asynchronous as the main types, with semi-synchronous as a possible extension.

© Hak Cipta Universiti Teknologi MARA

QUESTION 8 (10 marks)

Source: CSC429-2018-06, page [3]

Unofficial model answer:

a) Compute -19 - 99 = -118 in an 8-bit byte-addressable unit. Represent -19 and -99 in 8 bits, then perform subtraction as addition of the negated second operand.

i) Sign and magnitude: +19 = 0001 0011, so -19 = 1001 0011. +99 = 0110 0011, so -99 = 1110 0011. To subtract, add -19 + (+99) = 1001 0011 + 0110 0011 = 1 1111 0110 (9-bit). Discard the carry-out: 1111 0110. In sign-magnitude this is sign=1, magnitude=111 0110 = 118, so the result is -118. (Note: sign-magnitude addition/subtraction requires comparing magnitudes and choosing the sign; the direct binary addition happens to give the correct sign-magnitude pattern here, but the formal method is magnitude comparison.)

ii) One's and two's complement: +19 = 0001 0011; -19 in one's complement = 1110 1100; -19 in two's complement = 1110 1101. +99 = 0110 0011; -99 in one's complement = 1001 1100; -99 in two's complement = 1001 1101. For -19 - 99, add -19 and -99. One's complement: 1110 1100 + 1001 1100 = 1 1000 1000; end-around carry gives 1000 1001, which is -118 in one's complement (since 0111 0110 = 118). Two's complement: 1110 1101 + 1001 1101 = 1 1000 1010; discard carry gives 1000 1010, which is -118 in two's complement (since 1000 1010 = -128 + 10 = -118).

b) Compute -19 + 99 = +80 in an 8-bit byte-addressable unit.

i) Sign and magnitude: -19 = 1001 0011, +99 = 0110 0011. Since signs differ, subtract smaller magnitude from larger: 99 - 19 = 80, sign positive. Result = 0101 0000 (+80). Direct binary addition would give 1111 0110, which is not the correct sign-magnitude result; the magnitude-comparison rule must be applied.

ii) One's and two's complement: One's complement: -19 = 1110 1100, +99 = 0110 0011. Sum = 1 0101 1111; end-around carry gives 0101 1111 + 1 = 0110 0000? Wait, recompute: 1110 1100 + 0110 0011 = 1 0100 1111 (not 0101 1111). End-around carry: 0100 1111 + 1 = 0101 0000 = +80. Two's complement: -19 = 1110 1101, +99 = 0110 0011. Sum = 1 0101 0000; discard carry gives 0101 0000 = +80. No overflow (carry into sign equals carry out of sign).

Marking points:

- *1 mark: Correct 8-bit sign-magnitude representations of -19 and -99 (or +99) with sign bit and magnitude.*
- *1 mark: Correct sign-magnitude subtraction result -118 (1111 0110) with method (magnitude comparison or equivalent).*
- *1 mark: Correct 8-bit one's complement representations of -19 and -99.*
- *1 mark: Correct one's complement result -118 (1000 1001) including end-around carry handling.*
- *1 mark: Correct 8-bit two's complement representations of -19 and -99.*
- *1 mark: Correct two's complement result -118 (1000 1010) with carry discarded.*
- *1 mark: Correct sign-magnitude addition -19 + 99 = +80 (0101 0000) using magnitude comparison.*
- *1 mark: Correct one's complement addition result +80 (0101 0000) with end-around carry.*
- *1 mark: Correct two's complement addition result +80 (0101 0000) with carry discarded.*
- *1 mark: Clear working, correct byte width, and no overflow/range errors.*

Model confidence: high.

Model note: Assumes 8-bit byte-addressable unit as implied by 'byte-addressable unit'. Sign-magnitude arithmetic formally requires magnitude comparison; direct binary addition is shown only as a check. The one's complement intermediate in b(ii) was corrected during writing: 1110 1100 + 0110 0011 = 1 0100 1111, end-around carry gives 0101 0000. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 9 (10 marks)

Source: CSC429-2024-07, page [4]

Unofficial model answer:

© Hak Cipta Universiti Teknologi MARA

a) The icon in Figure 1 is the standard USB (Universal Serial Bus) trident/arrow symbol, so the interface is a USB interface — a serial, industry-standard external device interface used to connect peripherals (keyboards, mice, flash drives, printers, cameras) to the computer via a USB port and host controller.

b) A hot connection (hot plugging/hot swapping) means a device can be connected to or disconnected from the running system while power is on, without shutting down or rebooting the computer. The interface and OS must support detection and safe insertion/removal (e.g. USB's electrical design and enumeration), so the device is recognised and configured automatically at run time.

c) The three I/O techniques are: (1) Programmed I/O (polled I/O): the CPU issues the I/O command and repeatedly checks the device status register (busy/ready) in a polling loop, transferring data itself; the CPU is fully involved and wastes time waiting, so it is simple but inefficient. (2) Interrupt-driven I/O: the CPU starts the transfer and continues other work; when the device is ready it raises an interrupt, the CPU suspends its current program, runs the interrupt service routine to transfer data/handle the device, then resumes; CPU involvement is on demand rather than continuous, giving better utilisation but still per-byte/per-word CPU handling. (3) Direct Memory Access (DMA): a DMA controller is programmed by the CPU with source/destination and count, then moves blocks of data directly between the device and memory without CPU involvement per word; the CPU is only involved at setup and at completion (interrupt), so it is the most efficient for large block transfers.

Marking points:

• *2 marks (a): identifies USB interface (Universal Serial Bus) from the icon.*

• *2 marks (b): defines hot connection as connecting/disconnecting a device while the system is powered and running, without reboot, with automatic recognition.*

• *6 marks (c): 2 marks each for programmed I/O, interrupt-driven I/O and DMA, awarded for correct description plus explicit statement of the CPU's role/involvement in each.*

Model confidence: medium.

Model note: Figure 1 is supplied only by caption/extracted text; the answer assumes the icon is the standard USB symbol, which is the only interface icon consistent with the 'hot connection' clue in part (b). If the actual icon differs, part (a) and the framing of (b) would change. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 10 (10 marks)

Source: CSC429-2025-07, page [3]

Unofficial model answer:

Part (a) — Access methods (3 marks). The scenarios map to the three classic file/memory access methods: sequential, direct (random), and indexed. (i) *Read a C file from a folder* — this is **direct (random) access**: the file is located and opened by its name/path without reading preceding files, so the system jumps straight to the required block. (ii) *Access a file line by line / streaming a video frame by frame* — this is **sequential access**: records are read in order, one after another, from start to end (a stream must be consumed in sequence). (iii) *Access a specific file block using an index in a large file* — this is **indexed access**: an index table is searched to find the pointer to the desired block, which is then accessed directly. (iv) *Jump to a specific frame or timestamp in a video* — this is **direct/random access** (often supported by an index of timestamps), since the player seeks to an arbitrary position rather than scanning from the beginning. [Note: the extracted question text is garbled; the mapping above follows the standard CSC429 access-method taxonomy — sequential, direct/random, indexed — and the visible scenario keywords 'line by line', 'index', 'specific frame/timestamp'.]

Part (b) — Ranking by storage capacity (4 marks). From most to least capacity: **Tape > Hard drive > Main memory > L3 Cache > L1 Cache > Register.** Tape (magnetic tape) offers the largest capacity at lowest cost per bit; hard disk is next; main memory (RAM) is smaller than secondary storage; cache levels shrink as they get closer to the CPU (L3 larger than L1); registers are the smallest storage inside the CPU. This ordering also reflects the memory hierarchy: capacity increases as you move away from the processor while speed and cost

© Hak Cipta Universiti Teknologi MARA

per bit decrease.

Part (c) — Volatile memory (3 marks). *Volatile* means the memory **loses its stored contents when power is removed or interrupted**. Volatile memory (e.g., registers, cache, main memory/DRAM and SRAM) requires a continuous power supply to retain data, so its contents are temporary and must be reloaded from non-volatile secondary storage after a power cycle or reboot. Volatile memory is typically faster and is used to hold the currently executing program and data, whereas non-volatile memory (hard disk, SSD, ROM, tape) retains data without power and is used for permanent storage.

Marking points:

• 1 mark: (a) correctly identifies direct/random access for reading a named C file from a folder (with brief justification).

• 1 mark: (a) correctly identifies sequential access for reading a file line by line / streaming video in order.

• 1 mark: (a) correctly identifies indexed access for accessing a specific file block via an index, and direct/random access for jumping to a specific video frame/timestamp.

• 1 mark: (b) Tape ranked first (largest capacity).

• 1 mark: (b) Hard drive ranked second, above main memory.

• 1 mark: (b) Main memory > L3 Cache > L1 Cache correctly ordered.

• 1 mark: (b) Register ranked last (smallest) and full ordering correct: Tape > Hard drive > Main memory > L3 > L1 > Register.

• 1 mark: (c) defines volatile as losing contents when power is removed/interrupted.

• 1 mark: (c) gives examples of volatile memory (registers, cache, RAM/DRAM/SRAM) and states it needs continuous power.

• 1 mark: (c) contrasts with non-volatile memory (disk/SSD/ROM/tape) which retains data without power, and/or notes data must be reloaded after power loss.

Model confidence: medium.

Model note: The question text for part (a) is badly garbled in the extracted paper (OCR errors: 'Readi I files', 'Streami files line line', 'AS ecific file block usi an index', 'toas cific frame or a timestam'). The mapping of scenarios to access methods is inferred from the visible keywords and the standard CSC429 sequential/direct/indexed taxonomy; exact scenario wording could not be verified. Part (b) ranking assumes standard capacity ordering of the memory hierarchy. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2023-07, page [6]

Unofficial model answer:

Figure 3 represents the expression 242 − (B + AC), i.e. 242 minus the sum of register B and the product of registers A and C. Registers A, B and C are read-only; x, y and z are temporaries; the final result must end in register y. The three programs below use only the mnemonics listed in Table 1.

Zero-address program: With a stack machine, operands are pushed and the arithmetic instructions pop their operands and push the result. The constant 242 is pushed first, then B, then A and C are pushed and MUL is executed to leave AC on the stack; ADD then combines B and AC; SUB computes 242 − (B + AC); finally pop stores the top of stack into y. Program: PUSH 242; PUSH B; PUSH A; PUSH C; MUL; ADD; SUB; POP y. Comments: each PUSH places an operand on the stack; MUL replaces A and C by A*C; ADD replaces B and A*C by B+AC; SUB replaces 242 and (B+AC) by 242−(B+AC); POP y writes the result to y.

One-address program: A one-address machine uses an implicit accumulator. LOAD copies a memory/register operand into the accumulator; ADD, SUB, MUL and DIV combine the accumulator with the named operand and leave the result in the accumulator; STOR copies the accumulator to the named destination. Because A, B and C cannot be modified, the product AC is formed in a temporary: LOAD A; MUL C; STOR x (x = A*C); LOAD B; ADD x (accumulator = B + AC); STOR z (z = B + AC); LOAD 242; SUB z (accumulator = 242 − (B + AC)); STOR y. Comments: the first three instructions compute AC into x; the next two compute B + AC into z; the last three compute the final subtraction and store the answer in y.

Three-address program: Each three-address instruction has the form OP dest, src1, src2 and does not destroy its sources. Using temporaries x, y and z: MUL x, A, C (x = A*C); ADD z, B, x (z = B + AC); SUB y, 242, z (y = 242 − (B + AC)). Comments: MUL forms the product without changing A or C; ADD forms the parenthesised sum without changing B; SUB performs the final subtraction and places the required final answer directly in y. No STOR is needed because the destination of the last instruction is y.

Marking points:

• 2 marks: correctly identifies the expression as 242 − (B + AC) and respects the read-only constraint on A, B, C (uses only x, y, z as temporaries).

• 6 marks: zero-address program — correct use of PUSH/POP and stack order (2), correct MUL for AC (1), correct ADD for B+AC (1), correct SUB order 242 − (B+AC) (1), final POP y (1).

• 6 marks: one-address program — correct LOAD/STOR/accumulator usage (2), computes AC into a temporary without modifying A or C (1), computes B+AC (1), computes 242 − (B+AC) in the right order (1), final STOR y (1).

• 4 marks: three-address program — MUL x, A, C (1), ADD z, B, x (1), SUB y, 242, z (1), final answer in y and no source registers modified (1).

• 2 marks: correct assembly notation and meaningful comments throughout (award proportionally across the three programs).

Model confidence: high.

Model note: Figure 3 is supplied only as extracted text '242 - (B + AC)'; the answer assumes this is the intended expression. Table 1 lists only mnemonics, not operand syntax, so standard zero-, one- and three-address operand forms are assumed. The one-address column in the extracted table omits MUL; the answer assumes MUL is available on the one-address machine because the question requires computing AC. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA